# SeriCrypt: An LLM-Driven Context-Aware Serialization Framework for Cryptographic Protocols

Maosong Chen†, Xi Chen†✉, Mengcheng Ju†, Dongliang Zhao†, Chunxiang Gu†✉
†Information Engineering University, Zhengzhou 450001, China
Email: chenmaosong1214@163.com, xycuckoo@tsinghua.org.cn, zhangph12138@163.com, xiao__xiang@163.com, gcx5209@126.com



***Abstract*—Constructing syntactically correct and cryptographically valid message sequences is essential for protocol state machine learning, conformance testing, and fuzzing. Unlike plaintext protocols, cryptographic protocols involve complex cross-message state dependencies and cryptographic computation constraints. Existing automated message construction approaches predominantly target text-based or plaintext protocols, lacking support for deep semantic dependencies and cryptographic operations, rendering cryptographic protocol message construction largely manual. We present SeriCrypt, an LLM-driven, context-aware serialization framework for cryptographic protocols. SeriCrypt uses a large language model (LLM) to extract protocol field constraints, contextual state dependencies, and cryptographic computation rules from unstructured protocol specifications, representing them as a unified structured intermediate representation. To formally characterize this representation, we design a domain-specific language for cryptographic protocols (CDSL) that explicitly captures semantic constraints and computational dependencies in protocol interactions. Building upon CDSL, we implement a protocol-agnostic execution engine that parses CDSL declarations and automates field value resolution, cryptographic primitive invocation, and byte-stream serialization. As case studies of SeriCrypt in protocol security testing, we employ the framework to construct violation messages targeting specification-defined security constraints and validate its application in protocol fuzzing scenarios, with evaluation on multiple mainstream implementations of TLS 1.2/1.3, IKEv1/v2, SSH, and TLCP. Results show SeriCrypt can generate valid message sequences accepted by real implementations and successfully complete handshakes across all evaluated scenarios. Security constraint testing revealed five specification violations, and fuzzing reached deeper protocol states with higher code coverage than mainstream fuzzers under the same time budget, demonstrating the framework's practical value for cryptographic protocol security testing.**

***Index Terms*—cryptographic protocols, large language model, protocol serialization, domain-specific language, protocol security testing**

## I. Introduction

Cryptographic network protocols (e.g., TLS, IPsec, SSH) are the foundational infrastructure of modern network communication security, providing core functions such as confidentiality protection, identity authentication, and integrity verification. The security of these protocols depends not only on the completeness of the specification design but also on whether their implementations strictly adhere to the security constraints defined in the RFCs. Studies show that even when the specification itself is thoroughly verified, deviations at the implementation level can still lead to severe security vulnerabilities such as state machine bypass [1]–[4]. Therefore, security testing against protocol implementations is an indispensable means of ensuring communication security [5].

For encrypted protocols, constructing messages that conform to cryptographic specifications can effectively improve the efficiency of security testing methods such as fuzzing [6], state-machine learning [2], and conformance testing. This requires that messages not only satisfy precise byte-level encoding but also handle complex cryptographic computations within a complete session context, and that their values hold deep dependencies on the states of preceding interactions.

This strict requirement of cross-message state consistency and cryptographic correctness poses challenges for automating the construction of such messages. The knowledge required to construct such messages is described in natural language in protocol specifications (e.g., IETF RFCs and national standards). LLMs have natural language understanding capabilities and can automatically extract this protocol knowledge from the specification text, but two problems remain in their practical application: First, directly employing LLMs to generate binary messages or low-level handling code often fails to reliably meet the above constraints [7]. Second, although existing automation schemes have attempted to introduce formal protocol description languages to constrain LLMs in automatically extracting message layout, length, and nesting logic [8], [9], their research focus has primarily been on parsing and validating message syntactic structure, without covering cryptographic semantics such as key derivation, encryption encapsulation, and signature verification. As a result, even if developers can automatically obtain the syntactic skeleton and static attributes of messages, they still need to manually write substantial state maintenance and cryptographic computation code to construct valid messages that are actually accepted by the server.

To address this problem, this paper proposes SeriCrypt, a context-aware serialization framework for cryptographic protocols. The core idea of SeriCrypt is to decouple protocol structure description from cryptographic attribute computation. The LLM is only responsible for extracting a structured protocol intermediate representation—the Protocol Abstract Syntax Tree (PAST)—that conforms to the constraints of

CDSL. PAST organizes message formats in a hierarchical structure, and defines field lengths, cross-message references, key derivation, and encryption/decryption processes through attribute rules. Then the execution engine parses these attribute rules, calls underlying cryptographic primitives, and completes the evaluation and serialization. This design automates the construction of cryptographic protocol messages, and further allows testers to modify message structures at the PAST level to generate test sequences that conform to cryptographic specifications, thereby supporting security testing of protocol implementations.

We have implemented SeriCrypt and evaluated it on mainstream open-source implementations including OpenSSL, TLSE, strongSwan, Libreswan, OpenSSH, and GmSSL. The experiments cover six cryptographic protocols: TLS 1.2/1.3, IKEv1/v2, SSH, and TLCP. The results show that SeriCrypt can automatically construct acceptable legitimate message sequences and successfully complete full handshakes. In addition, we applied SeriCrypt to protocol security testing scenarios and discovered five security violations.

The contributions of this paper include:

- **Context-aware protocol representation.** We designed a domain-specific language for cryptographic protocols, CDSL, for uniformly describing message structures, field attributes, cross-message context references, and cryptographic computation rules such as key derivation, encryption, and authentication.
- **Specification-driven automatic extraction process.** We propose an LLM-based specification extraction chain-of-thought that extracts the Protocol Abstract Syntax Tree based on the LLM's existing protocol knowledge, reducing the cost of manually organizing protocol messages and cryptographic dependency rules.
- **Protocol-independent general execution engine.** We implemented a modular execution engine for CDSL attributes that completes field evaluation, cryptographic operation execution, and message serialization, so that protocol differences are mainly reflected in the CDSL description layer while the underlying execution mechanism can be reused across protocols.
- **Experimental validation and security testing application.** Through end-to-end evaluation of six mainstream implementations across six cryptographic protocols, we verified the interoperability of the messages generated by SeriCrypt for cryptographic protocols. Meanwhile, experiments demonstrate its supporting capability in protocol security testing scenarios, with five security violations discovered.

## II. Problem Definition and Design Requirements

The generation of cryptographic protocol messages is fundamentally different from that of plaintext protocols. For cryptographic protocols, even if the precise syntactic structure of protocol messages is known, if one cannot access and process the contextual information and cryptographic computations in the session process, it is still impossible to construct a message that can pass peer verification [10], [11]. The complexity is mainly reflected in two aspects: cross-message context references—some fields need to reference random numbers, negotiated parameters, and ephemeral keys exchanged or negotiated in previous messages; cryptographic computation constraints—some fields are themselves the results of cryptographic operations, such as key derivation, message authentication codes, and digital signatures.

Take the Finished message in TLS 1.3 as an example [12]. The syntactic structure of this message is simple, but the construction of its verify_data field must take as input the transcript-hash accumulated from previous handshake messages, combine it with the finished-key derived from the shared secret, and then compute the HMAC. Relying solely on static field format definitions cannot satisfy such dynamic computation requirements.

Accordingly, this paper decomposes the problem of automated construction of cryptographic protocol messages into two sub-problems:

- **Representation problem** — a formal language is needed to describe protocol message structures, cross-message state dependencies, and cryptographic computation logic, in order to automatically construct the protocol's intermediate representation.
- **Execution problem** — a general execution engine decoupled from specific protocols is needed, which can parse the aforementioned intermediate representation and drive the underlying cryptographic primitives to complete evaluation and serialization.

The system design needs to meet the following requirements:

- **R1** Structural expressiveness: The formal language must fully describe the organization of protocol messages, including field order, hierarchical relationships, length constraints, optional fields, and nested structures.
- **R2** Dependency expressiveness: The formal language must explicitly express the dependence of field values on the results of preceding interactions, making the sources of cross-message references traceable.
- **R3** Cryptographic computation expressiveness: The formal language must explicitly express the cryptographic computations involved in field assignments, concretizing the computation methods of relevant fields.
- **R4** Executability: The formal language must adopt a uniform, unambiguous formal definition, ensuring that the intermediate representation constructed under its constraints is machine-readable and can serve as input to the execution engine.
- **R5** Attribute-aware general parsing: The execution engine must have parsing capability independent of specific protocols, driving evaluation and serialization based on the attributes of nodes in the intermediate representation.

## III. Formal Model and Unified Attribute System

This section establishes the formal foundation of SeriCrypt. We propose the Context-Aware Attribute Grammar for Crypto-

graphic Protocols (CA-AGCP) model and, on this basis, define the Unified Attribute System (UAS).

### A. From CFG+AG to CA-AGCP

In classical formal language theory [13], a context-free grammar (CFG) is represented as a quadruple $G = (N, \Sigma, P, S)$. CFGs can naturally describe the hierarchical nested structure of protocol messages through recursive structures. Attribute grammars (AG) [14] introduce attribute sets $A$ and semantic rules $R$ on this basis, and achieve node evaluation through synthesized and inherited attributes. However, applying classical AGs to cryptographic protocols requires solving two additional modeling challenges: first, dynamic awareness of cross-message states — the field values of subsequent messages depend on the exchange results of previous messages, which extend beyond the local attribute propagation scope of standard AGs; second, primitive representation of cryptographic operators — key derivation, encryption/decryption encapsulation, and signature generation are usually composed of a series of cryptographic primitives, and the semantic rules of standard AGs are difficult to formally characterize such operations.These two challenges shape the extension design of CA-AGCP: a runtime context $\Gamma$ is introduced to maintain the cross-message state exchanged during the session, a cryptographic primitive space $\Omega$ to characterize cryptographic operations as ordered compositions of atomic primitives, complemented by a knowledge space $K$ supplying the protocol constants.

### B. Formal Definition of CA-AGCP

CA-AGCP is formally defined as a 9-tuple $G = (N, \Sigma, P, S, A, R, \mathrm{K}, \Gamma, \Omega)$, which can conceptually be divided into three layers: $\langle N, \Sigma, P, S\rangle$ forms the syntactic skeleton, defining the hierarchical basic topological structure of the protocol; $\langle A, R\rangle$ forms the basic attribute system, providing a node-level attribute evaluation framework; $\langle K, \Gamma, \Omega\rangle$ forms the dynamic semantic extension, which is the core increment of CA-AGCP relative to traditional attribute grammars.

**Definition 3.1 (Configuration and Knowledge Space).** The knowledge space K is defined as a finite total function from identifiers to constant values:

$$K : \text{ConstId} \to \text{Val}_k$$

where ConstId is the set of constant identifiers defined by the protocol specification (e.g., algorithm identifiers, version numbers, fixed label strings, etc.), and Val covers data types such as byte sequences and certificates. K is jointly constituted by the protocol specification, local security policies, and the current test configuration.

**Definition 3.2 (Runtime Context).** The runtime context $\Gamma$ is defined as a finite partial function from structured identifier paths to a value domain:

$$\Gamma : \text{KeySpace} \to \text{Val}$$

The key space contains two addressing modes: structured paths (used to locate field data in exchanged messages) and simple identifiers (used to name intermediate results produced during key derivation). $\Gamma$ supports a projection operation (read) and an update operation (write). The read operation projects the specified value from the context, and the write operation injects newly generated field values or intermediate keys into the session state.

**Definition 3.3 (Cryptographic Primitive Space).** The cryptographic primitive space $\Omega$ is defined as a set of cryptographic operations with a two-layer structure:

$$\Omega = \Omega_{\text{prim}} \cup \Omega_{\text{comp}}$$

where $\Omega_{\text{prim}}$ is the set of atomic primitives, each element directly mapping to a single algorithm call of the underlying cryptographic library (e.g., HMAC, AES-GCM-Encrypt, etc.); $\Omega_{\text{comp}}$ is the set of composite operations, consisting of ordered rule chains of atomic primitives and auxiliary functions according to data dependencies (e.g., key derivation, encryption/decryption encapsulation, etc.).

**Definition 3.4 (Protocol Abstract Syntax Tree).** Given a CA-AGCP grammar G, PAST is defined as a 2-tuple:

$$T_A = (T, B)$$

where T is the message structure tree, with each node carrying attribute annotations; B is the set of session-level computation blocks, each block consisting of an ordered rule chain of atomic primitives, auxiliary functions, and intermediate variables, used to generate key variables or define composite cryptographic operations. T and B are associated through the dynamic context $\Gamma$.

### C. Unified Attribute System (UAS)

To characterize the context dependencies and cryptographic transformations of cryptographic protocols, we construct the UAS, which divides the attribute set $A$ into the following three categories according to semantic function and computation method:

**Intrinsic Attributes ($A_I$):** Describe context-free atomic data units. Their attribute values are determined by the static semantics of the symbols themselves or local encoding rules, and do not depend on $\Gamma$ or $\Omega$. For an arbitrary symbol $X$, with its attribute $a \in A_I(X)$, the evaluation rule is formally expressed as:

$$\text{val}(a) = \Phi(X, K)$$

where $\Phi : (N \cup \Sigma) \times K \to \text{Val}$ is the independent evaluation function.

**Relational Attributes ($A_R$):** Represent data dependencies between protocol fields formed based on historical interactions. Their attribute values are highly dependent on the dynamic context $\Gamma$. For an arbitrary symbol $X$, with its attribute $a \in A_R(X)$, the evaluation rule is formally expressed as:

$$\text{val}(a) = \pi(\Gamma, \text{path}(a))$$

where $\pi$ is the state projection function, and $\text{path}(a) \in \text{KeySpace}$ is the context reference path declared by attribute $a$.

**Transformational Attributes ($A_T$):** Characterize the higher-order semantics driven by cryptographic primitives in the protocol, such as ciphertext generation, hash verification, and key derivation. For an arbitrary symbol $X$, with its attribute $a \in A_T(X)$, the evaluation rule is formally expressed as:

$$\text{val}(a) = \omega(\text{inputs}(a, \Gamma, K))$$

where $\omega \in \Omega$ is a specific cryptographic operation, and $\text{inputs}(a, \Gamma, K)$ is the tuple of input parameters extracted from the context $\Gamma$ and the knowledge space $K$ according to the declaration of attribute $a$.

## IV. CDSL: Domain-Specific Language for Cryptographic Protocols

CA-AGCP provides a formal foundation for the structural representation and semantic evaluation of cryptographic protocol messages. This section instantiates the model into an unambiguous, machine-parsable formal language — CDSL. The problem CDSL needs to solve is: how to simultaneously carry both the structural dimension and the semantic dimension from the 9-tuple within a unified grammatical framework. To this end, CDSL designs three subsystems: Message Structure Grammar, Unified Attribute Evaluation System, and Session-Level Cryptographic Computation Grammar. Examples of PAST in subsequent subsections can be found in Appendix A.

### A. Message Structure Grammar

The structural grammar of CDSL instantiates the syntactic framework $\langle N, \Sigma, P, S \rangle$ of CA-AGCP as a set of named production rules. The following BNF defines its abstract syntax:

$$\begin{aligned}
\langle \textit{Spec} \rangle &::= \text{`\{'}\ \langle \textit{RuleList} \rangle\ \text{`\}'} \\
\langle \textit{RuleList} \rangle &::= \langle \textit{Rule} \rangle \mid \langle \textit{Rule} \rangle\ \text{`,'}\ \langle \textit{RuleList} \rangle \\
\langle \textit{Rule} \rangle &::= \textit{RuleName}\ \text{`:'}\ \langle \textit{Alternatives} \rangle \\
\langle \textit{Alternatives} \rangle &::= \langle \textit{Branch} \rangle \mid \langle \textit{Branch} \rangle\ \text{`|'}\ \langle \textit{Alternatives} \rangle \\
\langle \textit{Branch} \rangle &::= \langle \textit{RefSeq} \rangle \mid \langle \textit{AtomicNode} \rangle \\
\langle \textit{RefSeq} \rangle &::= \langle \textit{RefNode} \rangle \mid \langle \textit{RefNode} \rangle\ \langle \textit{RefSeq} \rangle \\
\langle \textit{RefNode} \rangle &::= \text{`"name"'}\ \text{`:'}\ \textit{RuleName} \\
\langle \textit{AtomicNode} \rangle &::= \text{`"field\_length"'}\ \text{`:'}\ \langle \textit{LenSpec} \rangle\ \text{`,'} \\
&\qquad \text{`"value"'}\ \text{`:'}\ \langle \textit{AttrSpec} \rangle \\
\langle \textit{LenSpec} \rangle &::= \textit{length}\ \text{`:'}\ \text{`"'}\ \textit{INT 'bits'} \mid \textit{variable} \\
\langle \textit{AttrType} \rangle &::= \textit{static} \mid \textit{random} \mid \textit{local} \mid \textit{dependent} \\
&\qquad \mid \textit{length} \mid \textit{calculate}
\end{aligned}$$

The design of this grammar is derived from a systematic induction of cryptographic protocol specifications such as IKEv1/v2 and TLS 1.2/1.3 [12], [15]–[17]. Specifically, $\langle \textit{AtomicNode} \rangle$ maps to a terminal symbol $\sigma \in \Sigma$, representing the smallest indivisible data unit, where each instance carries both a length constraint ($\langle \textit{LenSpec} \rangle$) and an attribute annotation ($\langle \textit{AttrSpec} \rangle$); $\langle \textit{RefSeq} \rangle$ is composed of one or more $\langle \textit{RefNode} \rangle$ arranged in sequence, with each reference node pointing to the production of another rule via *RuleName*, expressed as $X \to Y_1, Y_2, \ldots, Y_k$, to represent the ordered concatenation of multiple substructures; $\langle \textit{Alternatives} \rangle$ models variable-length homogeneous sequences, corresponding to the recursive production $X \to YX \mid \epsilon$, whose termination condition is governed by local configuration.

### B. Unified Attribute Evaluation System

At the theoretical level, UAS classifies field attributes into three categories: $A_I$, $A_R$, $A_T$. CDSL instantiates this classification into six concrete attribute types, each carrying deterministic evaluation semantics.

Intrinsic Attributes $A_I$ have the syntax form:

$$\begin{aligned}
\langle \textit{IntrinsicAttr} \rangle ::= \textit{static(Value)} &\mid \textit{random(}\text{`"'}\ \textit{INT bits)} \\
&\mid \textit{local(ConfigKey)}
\end{aligned}$$

Among them, *static* binds a field to a definite value known from the protocol specification, resolved by the LLM based on the specification and the session configuration; *random* is generated by a cryptographically secure pseudo-random number generator according to the field length; *local* imports certificates, timestamps, etc., from local configuration.

Relational Attributes $A_R$ have the syntax form:

$$\begin{aligned}
\langle \textit{RelationalAttr} \rangle &::= \textit{dependent(RefPath)} \mid \textit{SimpleId} \\
\langle \textit{RefPath} \rangle &::= \textit{Ident}\ \ (\textit{Cat})\ \ \textit{[ByteRange]}
\end{aligned}$$

Among them, *dependent* references a specific field in previous messages or session state; *length* is an instance of synthesized attribute, whose value is obtained from the total number of bytes after serializing a specified subtree, backfilled by the execution engine after the evaluation of child nodes is completed.

Transformational Attributes $A_T$ have the syntax form:

$$\begin{aligned}
\langle \textit{CalcExpr} \rangle &::= \textit{calculate(}\langle \textit{PostLayer} \rangle\textit{)} \\
\langle \textit{PostLayer} \rangle &::= \langle \textit{CryptoLayer} \rangle \\
&\qquad \mid f_{\text{aux}}\ (\langle \textit{CryptoLayer} \rangle)\ (\ [\ \langle \textit{Range} \rangle\ ]\ )? \\
\langle \textit{CryptoLayer} \rangle &::= F\ (\langle \textit{Arg} \rangle (\text{`,'}\ \langle \textit{Arg} \rangle)^*) \\
\langle \textit{Arg} \rangle &::= (\langle \textit{Const} \rangle \mid \langle \textit{Var} \rangle \mid \langle \textit{FieldRef} \rangle \\
&\qquad \mid f_{\text{aux}}\ (\langle \textit{Arg} \rangle^+) \mid \langle \textit{CalcExpr} \rangle)(\text{`|'}\ \langle \textit{Arg} \rangle)^*
\end{aligned}$$

This structure contains four layers of semantics: the *calculate* wrapping layer marks the execution boundary of cryptographic computation; the auxiliary function layer ($f_{\text{aux}}$) performs non-cryptographic operations such as truncation, encoding conversion, and concatenation; the cryptographic function layer ($F$) specifies a specific algorithm call, which can be an underlying primitive function ($F_{\text{prim}}$) or a composite function defined in a computation block ($F_{\text{def}}$); the argument reference layer traces each argument back to constants, variables, field references, nested auxiliary functions, or recursion.

### C. Session-Level Cryptographic Computation Grammar

A single field's *calculate* expression can represent local cryptographic computation, but session-level computations in cryptographic protocols often include multi-step sequential

derivations. For example, TLS 1.3 needs to derive early_secret, handshake_secret, traffic_secret, and finished_key from the ECDHE shared secret [12]; IKEv2 needs to generate multiple key materials via prf+ [15]. To express such processes, CDSL introduces computation blocks, which are divided into flow-type and parameter-type.

Flow-type computation blocks are ordered composites of calculate expressions, arranging multiple assignment rules according to data dependencies into a sequentially executed rule chain. Its syntax is defined as follows:

$$\langle \textit{FlowBlock} \rangle ::= \textit{BlockName}\ \{\ \langle \textit{Rule} \rangle^{+}\ \}$$
$$\langle \textit{Rule} \rangle ::= \textit{Target} = \langle \textit{CalcExpr} \rangle \mid \langle \textit{DataExpr} \rangle$$
$$\langle \textit{DataExpr} \rangle ::= \langle \textit{AuxCall} \rangle \mid \langle \textit{ArgUnit} \rangle\ (\mid\ \langle \textit{ArgUnit} \rangle)^{*}$$

Here, $\langle \textit{CalcExpr} \rangle$ follows the same four-layer nested grammar as transformational attributes $A_T$, and $\langle \textit{DataExpr} \rangle$ is a pure data expression (concatenation, slicing, length calculation, etc.) not involving cryptographic operations. Rules within the block are evaluated strictly in the written order; after the *Target* of each rule is evaluated, it is injected into the context via $\Gamma$'s update operation, and subsequent rules can directly reference it via $\langle \textit{Var} \rangle$.

Parameter-type computation blocks describe algorithm configurations via declarative key-value pairs:

$$\langle \textit{ParamBlock} \rangle ::= \textit{BlockName}\ \{\ \langle \textit{Param} \rangle^{+}\ \}$$
$$\langle \textit{Param} \rangle ::= \textit{ParamKey}\ :\ (\langle \textit{Const} \rangle \mid \textit{null} \mid \textit{true} \mid \textit{false})$$

Each $\langle \textit{Param} \rangle$ corresponds to one configuration dimension for a key, used to declaratively define configuration parameters for operations such as signature algorithms.

### D. Comprehensive Example: TLS 1.3 Finished Message Construction

Using the TLS 1.3 Finished_client message as an example, we demonstrate how the three CDSL subsystems work together. Fig. 1 shows the complete flow from CDSL description to message generation for the Finished message:

1) SeriCrypt automatically generates the complete PAST under the constraints of CDSL, comprising the message structure tree $T$ carrying node attribute annotations and the set of session-level computation blocks $B$.
2) The execution engine parses the nodes of the PAST. When it reaches the calculate expression of the verify_data field in the Finished_client message, a transformational attribute, it recognizes that the evaluation of verify_data depends on handshake_messages and client_handshake_traffic_secret, and accordingly triggers the KeyDerivation and PrfDef computation blocks.
3) KeyDerivation reads the DH key, ClientHello, ServerHello, and the preceding handshake messages from the runtime context $\Gamma$, sequentially derives keys such as early_secret and handshake_secret, and writes the results back to $\Gamma$.
4) The calculate expression of verify_data reads the required key from $\Gamma$, invokes hkdf_expand_label, defined in the PrfDef computation block, to derive finished_key, and computes verify_data via HMAC.
5) The execution engine completes encryption encapsulation, length backfilling, and byte-level serialization, ultimately producing a Finished_client message that conforms to the TLS 1.3 specification.

Throughout this process, cross-message dependencies are resolved through attribute declarations and projections from the runtime context, and the computation blocks, once defined, are reused by the execution engine without protocol-specific code.

```
Finished_client {
  "Finished_client": [[{"name": "record_layer"},{"name": "Encrypted_block"}]],
  "Encrypted_block": [[{"name": "handshake_layer"},{"name": "message_body"}]],
  "message_body": [[{"name": "verify_data"}]],
  "verify_data": [[{"field_length":"32bits","value":"calculate(truncate(hmac(
                hkdf_expand_label(client_handshake_traffic_secret, "finished", "",32),
                hash(ClientHello_client(handshake_messages)[:] | ... |
                CertificateVerify_server(handshake_messages)[:])))[0:32])"}]]
}
Key_Derivation {
   early_secret = calculate(hkdf_extract(repeat(0x00, 32), repeat(0x00, 32)))
   derived_early_secret = calculate(hkdf_expand_label
                              (early_secret, "derived", calculate(hash("")), 32))
   shared_secret = calculate(dh(local(supported_group),
                              ServerHello_server(handshake_messages)[-256:]))
   handshake_secret = calculate(hkdf_extract
                              (derived_early_secret, shared_secret))[0:32]
   hello_transcript_hash = calculate(hash(ClientHello_client(handshake_messages)[:]
                              | ServerHello_server(handshake_messages)[:]))
   client_handshake_traffic_secret = calculate(hkdf_expand_label(handshake_secret,
                               "c hs traffic", hello_transcript_hash, 32))[0:32]
   server_handshake_traffic_secret = calculate(hkdf_expand_label(handshake_secret,
                              "s hs traffic", hello_transcript_hash, 32))[0:32]
   client_handshake_key = calculate(hkdf_expand_label
                              (client_handshake_traffic_secret, "key", "", 16))[0:16]
   client_handshake_iv = calculate(hkdf_expand_label
                              (client_handshake_traffic_secret, "iv", "", 12))[0:12]
   server_handshake_key = calculate(hkdf_expand_labe
                              l(server_handshake_traffic_secret, "key", "", 16))[0:16]
   server_handshake_iv = calculate(hkdf_expand_label
                              (server_handshake_traffic_secret, "iv", "", 12))[0:12]
}
Prf_Def{
    hkdf_expand_label(Secret, Label, Context, Length)
                        = hmac(Secret, uint16(Length)| uint8(length("tls13 " | Label)) |
                        "tls13 " | Label | uint8(length(Context)) | Context | 0x01)
}
```

Fig. 1: The Finished_client message part in PAST.

## V. System Design and Implementation

### A. System Overview

SeriCrypt divides the construction of cryptographic protocol messages into two phases: Phase1—LLM-driven PAST construction, and Phase2—attribute-driven PAST execution, with CDSL serving as the formal bridge connecting the two phases. Moreover, the declarative structure of PAST supports protocol security testing: targeted mutations at the PAST level can construct specification-violating test inputs for violation detection (RQ5) and structured test cases for protocol fuzzing

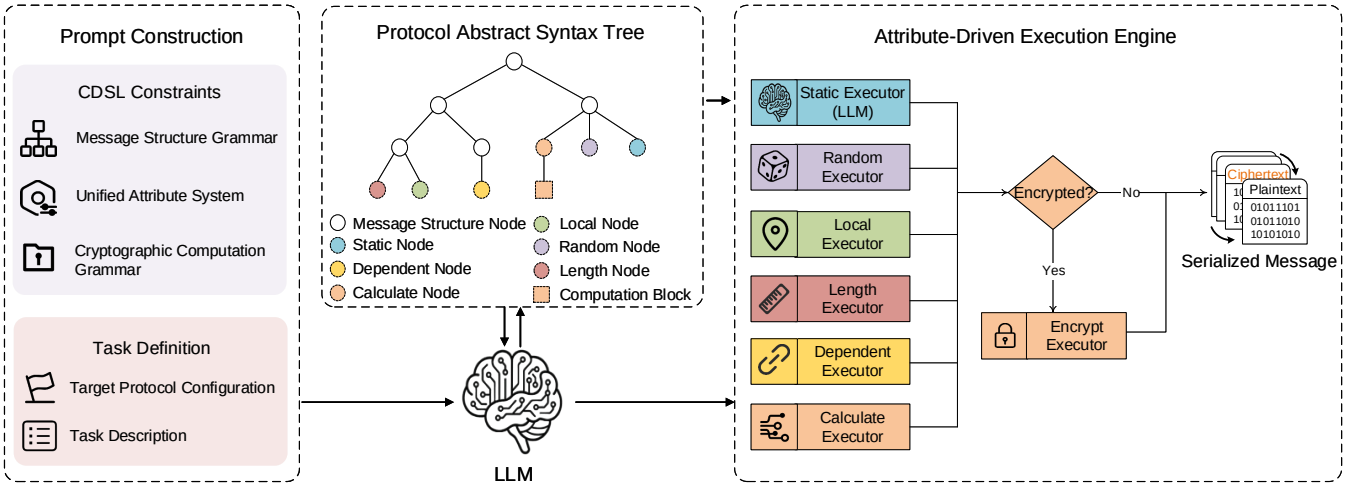


Fig. 2: SeriCrypt system operation flow.

(RQ6). Fig. 2 illustrates the full operation flow of the SeriCrypt system.

### *B. Phase1: LLM-Driven PAST Construction*

PAST construction adopts a chain-of-thought strategy [18], [19], decomposing the complex specification understanding task into three stages comprising eight sequential steps, each of which takes the output of the preceding step as input (complete prompt templates are given in Appendix B):

1) **Message structure construction** — the LLM generates the packet composition of each client message following the protocol encapsulation hierarchy, and transforms it into grammar productions carrying field length constraints and initial attribute annotations;
2) **Attribute annotation** — the LLM supplements each atomic field with attribute types and value rules according to the CDSL unified attribute evaluation system, and through dependency tracing refines the value rules into references to concrete message fields;
3) **Cryptographic block definition extraction** — the LLM extracts the key derivation formulas, encryption flow, decryption flow, signature parameter configurations, and specialized PRF functions according to the protocol specification and CDSL session-level cryptographic computation grammar, and writes them into corresponding computation blocks.

To address errors that LLM hallucinations may introduce into the PAST, SeriCrypt sets up a verification loop: the generated messages are checked by the execution engine, and any PAST that fails the check is corrected by humans against the protocol specification field by field until the verification passes, thereby preventing errors in the extraction stage from propagating to downstream testing.

### *C. Phase2: Attribute-Driven PAST Execution*

The execution engine is responsible for converting PAST into a sendable byte stream. The engine does not embed any protocol-specific message format or composite algorithm; its behavior is entirely driven by the structure rules, attribute declarations, and computation blocks in PAST, executed in the following three stages:

1) **Tree instantiation and independent field evaluation** — the engine expands the syntax tree instances in depth-first order according to the production set, solving static attributes (taken by LLM based on protocol knowledge), random attributes (generated by the underlying random number generator), and local attributes (value read from local files);
2) **Dependent field and cryptographic computation evaluation** — processes dependent (project value from $\Gamma$), length (sum subtree serialization length), and calculate (recursive expression engine evaluation);
3) **Encryption encapsulation and serialization** — loads the encryption computation block, replaces the plaintext subtree with ciphertext leaf nodes, then traverses depth-first, concatenating the byte values of all leaf nodes to output the final message.

Protocol independence is a key design of the execution engine. The engine only embeds a set of atomic cryptographic primitives (hmac, dh, aes_gcm_encrypt, etc.) and does not provide protocol-specific composite algorithms. Composite functions in protocol specifications (such as TLS 1.3's HKDF-Expand-Label, IKEv2's prf+) are extracted by Phase1 as parameterized definitions and written into computation blocks. At runtime, the engine loads these computation blocks, expands the composite expressions into atomic primitives, and then uniformly evaluates them via the recursive expression engine. Protocol differences are thus encapsulated in the CDSL description layer, and no modification to the execution engine code is needed when extending to new protocols.

## VI. Experimental Evaluation

We evaluate SeriCrypt on six cryptographic protocols across four protocol families. The evaluation revolves around the following research questions:

- **RQ1 (Interoperability):** Can SeriCrypt automatically generate handshake messages that are accepted by real protocol implementations?
- **RQ2 (Expressiveness):** Can CDSL's attribute system and computation block mechanism fully describe all message structures and cryptographic operations that the client needs to construct in the target protocols?
- **RQ3 (LLM Generation Quality):** What is the accuracy of PAST generated by the LLM?
- **RQ4 (LLM Comparison):** Can directly prompting the LLM to generate a complete protocol client successfully complete the handshake?
- **RQ5 (Protocol Testing Application):** How practical is SeriCrypt in protocol testing?
- **RQ6 (Fuzzing Application):** How does SeriCrypt's cryptographic protocol serialization capability perform in protocol fuzzing scenarios?

### A. Experimental Setup

**Protocols and implementations.** The experiments cover 10 protocol configurations: IKEv1 (PSK/certificate, Main Mode/Aggressive Mode), IKEv2 (PSK/certificate), TLS 1.2, TLS 1.3, SSH (password authentication), and TLCP (dual-certificate mutual authentication), tested on six mainstream open-source implementations: strongSwan 5.9.14, Libreswan 4.12, OpenSSL 3.0.2, TLSe (commit e69790b), OpenSSH 9.8p1, and GmSSL 3.1.1.

**Verification method.** In protocol-testing scenarios, this work supports obtaining legitimate message sequences whose fields conform to the specification and that are accepted by real implementations as valid protocol inputs. To determine whether a generated message satisfies this requirement, we adopt two criteria: (1) the server accepts the message and returns the response prescribed by the normal protocol negotiation flow; and (2) Wireshark field-level parsing reveals no anomaly in the message.

**Auxiliary components.** Each protocol is equipped with a configuration file and a lightweight response parser of approximately 120 lines, the latter extracting the field values required for subsequent computations from the server's responses.

### B. RQ1: Interoperability

For each combination of protocol, server, and cipher suite, we run SeriCrypt to complete the full interaction.

SeriCrypt successfully completed handshakes in all 10 test configurations (Table I), and all messages passed Wireshark field-level parsing. The server accepted the negotiation parameters, key materials, authentication fields, and encrypted payloads in the generated messages, demonstrating the interoperability of SeriCrypt in automatically constructing cryptographic protocol messages.

### C. RQ2: Expressiveness

We evaluate CDSL's expressiveness from two aspects: message structure and cryptographic operations. At the message structure level, CDSL described a total of 52 client messages under 10 protocol configurations, with IKEv1, IKEv2, TLS 1.2, TLS 1.3, SSH, and TLCP accounting for 18, 12, 6, 5, 5, and 6 messages, respectively. Every field could be mapped to one of CDSL's six attribute types, with no field pattern falling outside the attribute system's coverage. At the cryptographic operation level, the six protocols involved a total of 28 different cryptographic operations (see Table II for details), all of which could be declared through CDSL computation blocks.

### D. RQ3: LLM Generation Accuracy for PAST

For each target message in the six protocols, we used the Phase1 pipeline with two LLMs, Gemini-3-Pro-Preview and ChatGPT-5.5, each of which repeatedly generated PAST five times. The accuracy of each stage was measured on the generated PAST before correction: during the manual verification described in Section V-B, every generated PAST was reviewed against the protocol specification, the errors found in the LLM output were recorded, and the accuracy was computed based on the correctly generated items.

Both LLMs exhibit similar error characteristics. As shown in Table III, the structure generation accuracy of both LLMs is the lowest for TLS 1.3, because TLS 1.3 moves many functions into ClientHello extensions, which are numerous, deeply nested, and subject to complex optional conditions, so the LLM easily misjudges which extensions must appear. The attribute annotation accuracy of IKEv1/v2 and TLS 1.3 is relatively low, as their field values are scattered across specific payloads of multiple exchanges, making value tracing difficult. The computation block extraction accuracy of SSH is the lowest, because its key derivation adopts a character-labeled hash chain, whose operand order and splitting details are error-prone. We analyzed the errors recorded during the manual verification of both LLMs across the six protocols, and the errors of both models fall into the same three categories. **E1 (misjudgment of conditional field presence):** optional fields are numerous, and the prompts cannot precisely cover every optional part, so their presence is easily misjudged. **E2 (cross-message state and transcript boundary):** when a field value depends on complex message context, dependency tracing sometimes cannot reach the exact source location. **E3 (cryptographic derivation detail error):** the overall structure of the derivation is correct, but the operand order, trailing bytes, or splitting layout is wrong. Overall, although the above errors commonly occur in LLM-generated PAST, the average accuracy remains above 94%, and these errors can all be discovered and corrected through the verification loop described in Section V-B, demonstrating the value of the LLM in the SeriCrypt framework.

TABLE I: Handshake completion results.

| Protocol | Auth Mode | Exchange Mode | Cipher Suite | Handshake | Wireshark |
|---|---|---|---|---|---|
| IKEv1 | PSK | Main Mode | aes128-sha1-modp1536 | ✓ | ✓ |
| IKEv1 | PSK | Aggressive Mode | aes128-sha1-modp1536 | ✓ | ✓ |
| IKEv1 | Certificate | Main Mode | aes128-sha1-modp1536 | ✓ | ✓ |
| IKEv1 | Certificate | Aggressive Mode | aes128-sha1-modp1536 | ✓ | ✓ |
| IKEv2 | PSK | — | aes128gcm16-sha1-modp1536 | ✓ | ✓ |
| IKEv2 | Certificate | — | aes128gcm16-sha1-modp1536 | ✓ | ✓ |
| TLS 1.2 | Certificate | — | TLS_DHE_RSA_WITH_AES_128_GCM_SHA256 | ✓ | ✓ |
| TLS 1.3 | Certificate | — | TLS_AES_128_GCM_SHA256 | ✓ | ✓ |
| SSH | Password | — | dh-group14-sha256-aes128-gcm | ✓ | ✓ |
| TLCP | Certificate | — | ECDHE_SM4_GCM_SM3 | ✓ | ✓ |

TABLE II: Cryptographic operations covered by CDSL computation blocks.

| Protocol | Key Derivation | Symmetric Encryption | Symmetric Decryption | Digital Signature | Pseudo-random Function |
|---|---|---|---|---|---|
| IKEv1 | SKEYID_d/a/e | Enc_IKEv1 | Dec_IKEv1 | RSA | PRF |
| IKEv2 | SK_d/ai/ar/ei/er/pi/pr | Enc_IKEv2 | Dec_IKEv2 | RSA-PKCS1v15 | PRF+ |
| TLS 1.2 | Write_MAC_Key, Write_Key/IV | Enc_TLS1.2 | Dec_TLS1.2 | RSA-PKCS1v15 | TLS1.2_PRF (P_hash) |
| TLS 1.3 | Handshake_Key/IV | Enc_TLS1.3 | Dec_TLS1.3 | RSA-PSS | Hkdf_Expand_Label |
| SSH | Enc_Key/IV | Enc_SSH | Dec_SSH | – | – |
| TLCP | Write_Key/IV | Enc_TLCP | Dec_TLCP | SM2 | TLCP_PRF (P_SM3) |

TABLE III: PAST generation accuracy in each stage.

| Protocol | LLM | Structure | Attribute | Computation Block |
|---|---|---|---|---|
| IKEv1 | Gemini-3-Pro-Preview | 98.6% | 94.9% | 93.1% |
| IKEv1 | ChatGPT-5.5 | 94.2% | 92.8% | 92.4% |
| IKEv2 | Gemini-3-Pro-Preview | 95.5% | 93.9% | 95.8% |
| IKEv2 | ChatGPT-5.5 | 94.1% | 91.3% | 92.5% |
| TLS 1.2 | Gemini-3-Pro-Preview | 98.7% | 94.8% | 95.8% |
| TLS 1.2 | ChatGPT-5.5 | 97.4% | 95.1% | 97.9% |
| TLS 1.3 | Gemini-3-Pro-Preview | 92.1% | 93.3% | 98.0% |
| TLS 1.3 | ChatGPT-5.5 | 93.4% | 93.4% | 95.5% |
| SSH | Gemini-3-Pro-Preview | 96.7% | 96.7% | 92.9% |
| SSH | ChatGPT-5.5 | 95.1% | 95.0% | 91.6% |
| TLCP | Gemini-3-Pro-Preview | 96.1% | 98.0% | 94.7% |
| TLCP | ChatGPT-5.5 | 96.2% | 96.2% | 94.4% |

### *E. RQ4: Comparison with Direct LLM Generation*

We adopt "direct LLM generation" as the comparison baseline, i.e., asking the same LLM to directly generate runnable protocol client code; the LLM used for this baseline is Gemini-3-Pro-Preview. To test this baseline, we designed a progressive prompting strategy to generate a complete protocol client for each target protocol. The prompting process decomposes the task into steps such as message structure definition, structure construction and serialization, key derivation, encrypted payload construction, and interaction flow control. Each target protocol was repeated 5 times, for a total of 30 sets of experiments. The generated results were classified into five levels (L0 — code cannot run, L1 — message rejected, L2 — partial plaintext negotiation completed, L3 — encryption construction failed, L4 — full handshake successful).

TABLE IV: Direct LLM generation results.

| Protocol | Best Level | Main Failure Cause |
|---|---|---|
| IKEv1 | L2 | Key derivation failure |
| IKEv2 | L2 | Negotiation material acquisition failure |
| TLS 1.2 | L3 | Encrypted payload construction failure |
| TLS 1.3 | L2 | Key derivation failure |
| SSH | L2 | Key derivation failure |
| TLCP | L2 | Key derivation failure |

In all 30 sets of experiments, none reached L4 (full handshake successful) (Table IV). We categorized the observed failures into three types: (1) Protocol constant or encoding convention errors — the LLM can identify the general structure of message types and fields but easily miswrites constant values, byte order, and length encoding. (2) Historical message negotiation value acquisition failure — the code generated by the LLM often fails to stably maintain cross-message state, potentially ignoring the cipher suite selected by the server or omitting random numbers. (3) Cryptographic computation chain breakage — key derivation and encryption encapsulation are the primary failure sources; the LLM easily confuses HKDF label concatenation format, PRF input order, and the scope of AEAD additional authenticated data.

These results corroborate SeriCrypt's division-of-labor strategy. The LLM excels at extracting protocol knowledge from protocol specifications (RQ3 indicates that the average accuracy of PAST generation exceeds 94%). CDSL's design precisely separates specification extraction from computation execution: the LLM handles specification understanding and knowledge extraction, which it is relatively good at; the execution engine handles precise computation and byte-level encoding, which the LLM struggles to perform stably. This conclusion echoes existing work [7], [8], [20], [21]; the experimental results further show that when the task boundary extends from format extraction to cryptographic computation, the LLM's reliability rapidly declines. The combination of declarative intermediate representation and deterministic execution engine can mitigate this capability gap.

### *F. RQ5: Protocol Testing Application*

We extracted security constraints identified by normative keywords such as MUST, MUST NOT, SHOULD, and SHOULD NOT [22] (and their equivalents in the TLCP national standard [23]) from the specifications of the six target protocols, selected the subset that could be violated via field-level PAST mutations, and guided the LLM to construct

multiple test cases for each constraint. During testing, the system first executes all messages before the target message normally according to the original PAST to establish the session context, and then loads and sends the mutated PAST at the target message. Mutations are expressed at the declarative PAST level while encoding is still handled by the execution engine, so the mutated messages remain syntactically correct but violate the targeted constraints, thus reaching the server's actual security check logic; if the server does not reject as the specification requires, a potential violation is flagged [24].

TABLE V: Security constraint testing results.

| Protocol | Security Constraints | Test Cases | Security Violations |
|---|---|---|---|
| IKEv1 | 27 | 66 | 0 |
| IKEv2 | 26 | 79 | 1 |
| TLS 1.2 | 20 | 54 | 0 |
| TLS 1.3 | 16 | 61 | 4 |
| SSH | 19 | 55 | 0 |
| TLCP | 17 | 49 | 0 |
| Total | 125 | 364 | 5 |

Based on the 125 security constraints extracted from the specifications, 364 test cases were constructed, and 5 security violations were discovered (Table V), of which 4 were new findings . A detailed analysis of the security violations is provided in Appendix C. The results indicate that PAST's declarative structure can support targeted testing of protocol security constraints, verifying the practicality of SeriCrypt in protocol testing scenarios.

The five violations (Table VI) are concentrated in certificate verification, extension checking, and session maintenance, which indicates that implementations tend to relax the enforcement of RFC security constraints at these checkpoints; V-1 was disclosed in prior work, while V-2 through V-5 are new findings of this work and have been reported to the developers.

### G. RQ6: Fuzzing Application

To evaluate SeriCrypt's performance in protocol fuzzing scenarios, we augment SeriCrypt with a basic structured mutation component, which performs structured mutations on PAST fields to generate test cases. We compare SeriCrypt with the mainstream fuzzing tools AFLnet [25] and ChatAFL [20] on six cryptographic protocols — IKEv1, IKEv2, TLS 1.2, TLS 1.3, SSH, and TLCP — and measure the code coverage achieved within the same time budget of four hours.

SeriCrypt achieves the highest code coverage on all six protocols (Table VII). These protocols impose strict requirements on the cryptographic correctness of messages, and most blind byte-level mutations fail to pass the server's cryptographic checks, whereas PAST-based structured mutations preserve cryptographic validity, enabling the test cases to reach deeper protocol states. The results indicate that SeriCrypt's cryptographic protocol serialization capability effectively supports protocol fuzzing, and that the time overhead of dynamically parsing declarations and evaluating the attribute tree does not impact overall throughput, as SeriCrypt achieves the highest coverage within the same time budget.

## VII. Discussion

### A. Why CDSL

The structural description layer of CDSL overlaps with typed JSON schemas and with format description languages such as 3D [26] and Kaitai [27], all of which declaratively express field types, length constraints, and nesting levels, and differs from them only in where the declarative boundary is drawn. JSON Schema draws this boundary at the logical-structure level: it cannot represent binary encoding, so the binary encoding, field dependencies, and cryptographic semantics required for cryptographic protocol modeling are all undertaken by accompanying code; Kaitai and 3D push the boundary further, to binary layout and intra-message computed fields. Yet what lies beyond the boundary, cross-message session state and cryptographic semantics, is precisely the core of cryptographic protocols, and no modest extension of the intra-message expressive model of existing languages can cover these two requirements, which can only be realized in code. CDSL pushes the declarative boundary further still, to cross-message state and cryptographic semantics, and through reusable computation blocks decomposes protocol-specific composite algorithms into atomic cryptographic primitives, achieving protocol independence on the basis of the representation of cryptographic protocols.

### B. Extensibility

The favorable extensibility to new protocols is a significant advantage of SeriCrypt over existing methods such as Scapy [28] and TLS-Attacker [29]. The current evaluation covers the TLS, IKE, SSH, and TLCP protocol families which differ significantly in key derivation, message encapsulation, and exchange mechanisms. The results of RQ2 show that CDSL's six attribute types and computation block mechanism have sufficient expressiveness for these differences. When extending SeriCrypt to new protocols, the core logic of the execution engine is fully reused, and the manual effort consists of three parts: (1) constructing a configuration file that specifies session parameters such as the addresses of both endpoints, the selected algorithm types, and the message types to be generated, together with an output example of 5–8 fields; (2) implementing a lightweight response parser of approximately 120 lines; (3) manually verifying and correcting the LLM-generated PAST.

### C. Future Directions

SeriCrypt currently focuses on the serialization of client messages. Automatically generating response parsers based on the CDSL framework is the next focus of work. Meanwhile, we plan to further validate the extensibility of SeriCrypt on a broader range of cryptographic protocols (e.g., QUIC [30], WireGuard [31], and post-quantum hybrid handshakes) and diverse cryptographic algorithm combinations. These protocols often have more complex specifications or less coverage in the training data, so the error types summarized in RQ3 may become more pronounced, and the verification loop described in Section V-B will play a more important role. Furthermore,

TABLE VI: Summary of security violations.

| ID | Implementation (Protocol) | Violation | Security Impact |
|---|---|---|---|
| V-1 | strongSwan (IKEv2) | No response to INFORMATIONAL request | Session maintenance silently interrupted |
| V-2 | TLSE (TLS 1.3) | Use of non-negotiated signature algorithm | Negotiation constraint broken, risking weak algorithm selection |
| V-3 | TLSE (TLS 1.3) | Failure to reject MD5-signed certificate | Weak-hash certificate passes authentication |
| V-4 | TLSE (TLS 1.3) | Failure to check required extension | Handshake proceeds without a required extension |
| V-5 | TLSE (TLS 1.3) | Acceptance of illegal ChangeCipherSpec value | Malformed message silently ignored |

TABLE VII: Code coverage (AFL bitmap %) achieved under 4-hour fuzzing.

| Protocol (Implementation) | SeriCrypt | AFLnet | ChatAFL |
|---|---|---|---|
| IKEv2 (strongSwan) | 29.38% | 23.14% | 22.85% |
| IKEv1 (strongSwan) | 27.51% | 19.05% | 19.01% |
| TLS 1.3 (OpenSSL) | 24.87% | 23.81% | 23.83% |
| TLS 1.2 (OpenSSL) | 25.31% | 21.72% | 21.69% |
| SSH (OpenSSH) | 8.19% | 6.62% | 6.60% |
| TLCP (GmSSL) | 12.05% | 9.11% | 9.02% |

combining PAST mutation strategies with structure-aware fuzzing [21], [32], [33] is a natural evolution direction, and the preliminary results of RQ6 support this direction. Another promising direction is to replace the manual verification of LLM-generated PAST with agents (e.g., Claude Code) that iteratively test and repair the PAST, further reducing the human effort.

## VIII. Related Work

Table VIII compares the capability boundaries of SeriCrypt with representative approaches across four dimensions: D1 (automated specification knowledge extraction), D2 (cross-message dependency expression), D3 (cryptographic computation expression), and D4 (protocol-independent execution engine). ChatAFL [20] uses LLMs to extract protocol state machines from RFCs to guide fuzzing. 3DGen [8] combines LLMs with format description languages to generate verified binary format parsers. ParCleanse [9] extracts DSL-format constraints from RFCs and combines them with the Z3 solver to generate constraint-satisfying packets. SemFuzz [21] extracts constraints from RFCs to implement structured mutations on plaintext messages. These works demonstrate that LLMs can effectively participate in extracting protocol format knowledge and state machine knowledge, but their downstream tasks do not involve cross-message cryptographic state maintenance. TLS-Attacker [29] and Scapy [28] provide byte-level message construction capabilities, but the cryptographic logic is written in imperative code, and supporting each new protocol or cipher suite requires writing substantial protocol-specific code, making cross-protocol reuse difficult;in protocol security testing, constructing semantic-level mutations such as malformed messages that involve cross-message dependencies or cryptographic computations likewise requires locating and modifying the corresponding computation logic layer by layer, at a substantially higher implementation cost than SeriCrypt. We quantitatively verify this difference with a controlled experiment (see Appendix D for details).

TABLE VIII: Comparison between SeriCrypt and existing methods.

| Method | D1 | D2 | D3 | D4 |
|---|---|---|---|---|
| TLS-Attacker | × | ○ | ○ | × |
| Scapy | × | ○ | ○ | × |
| ChatAFL | ✓ | ○ | × | – |
| 3DGen | ✓ | × | × | ✓ |
| ParCleanse | ✓ | × | × | ✓ |
| SemFuzz | ✓ | × | × | ✓ |
| SeriCrypt | ✓ | ✓ | ✓ | ✓ |

*Note:* ✓ = support, × = no support, ○ = partial support, – = not applicable.

## IX. Conclusion

This paper proposes SeriCrypt, a context-aware serialization framework for network cryptographic protocols. SeriCrypt designs a domain-specific language for cryptographic protocols, CDSL, which uniformly describes message structures, field attributes, cross-message context references, and cryptographic computation rules. It proposes an LLM-based specification extraction process that converts unstructured specification text into PAST. It implements a protocol-independent general execution engine that, driven by attribute declarations, completes field evaluation and message serialization. Experimental results show that SeriCrypt can generate valid messages accepted by real protocol implementations, completing handshakes in all 10 test configurations. Directly prompting the LLM to generate client code failed to achieve a single complete handshake. The average accuracy of LLM-generated PAST exceeds 94%. Based on PAST, targeted violation tests against security constraints successfully constructed test cases and discovered 5 security violations, verifying the framework's practicality in protocol security testing scenarios. In protocol fuzzing, SeriCrypt can generate test sequences that pass the server's cryptographic checks, enabling fuzzing to reach deeper protocol states. The above results indicate that the division-of-labor strategy of "LLM extracts structure, deterministic engine executes computation" is applicable to scenarios such as cryptographic protocols that impose strict requirements on state consistency and computational correctness. The cryptographic protocol message construction capability provided by SeriCrypt can further serve protocol security testing tasks such as fuzzing, state machine learning, and conformance checking.

## Acknowledgment

We would like to thank our shepherd and the anonymous reviewers for their thoughtful feedback. This work is supported by the National Key Research and Development Program of China under Grant 2023YFA1009500.

## REFERENCES


[1] P. Fiterau-Brostean, B. Jonsson, R. Merget, J. De Ruiter, K. Sagonas, and J. Somorovsky, "Analysis of DTLS implementations using protocol state fuzzing," in *29th USENIX Security Symposium (USENIX Security 20)*, 2020, pp. 2523–2540.

[2] P. Fiterau-Brostean, B. Jonsson, K. Sagonas, and F. Tåquist, "Automata-based automated detection of state machine bugs in protocol implementations," in *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2023.

[3] F. Bäumer, M. Maehren, M. Brinkmann, and J. Schwenk, "Finding SSH strict key exchange violations by state learning," in *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security*, 2025, pp. 246–260.

[4] F. Bäumer, M. Brinkmann, and J. Schwenk, "Terrapin attack: Breaking SSH channel integrity by sequence number manipulation," in *33rd USENIX Security Symposium (USENIX Security 24)*, 2024, pp. 7463–7480.

[5] D. Zhao, J. Guo, C. Gu, Y. Zheng, and X. Zhang, "AGLFuzz: Automata-guided fuzzing for detecting logic errors in security protocol implementations," *Computers & Security*, vol. 149, p. 103979, 2025.

[6] M. Ammann, L. Hirschi, and S. Kremer, "DY fuzzing: Formal Dolev-Yao models meet cryptographic protocol fuzz testing," in *2024 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2024, pp. 1481–1499.

[7] Y. Li, Q. Pei, M. Sun, H. Lin, C. Ming, X. Gao, J. Wu, C. He, and L. Wu, "CipherBank: Exploring the boundary of LLM reasoning capabilities through cryptography challenge," in *Findings of the Association for Computational Linguistics: ACL 2025*, 2025, pp. 5929–5965.

[8] S. Fakhoury, M. Kuppe, S. K. Lahiri, T. Ramananandro, and N. Swamy, "3DGen: AI-assisted generation of provably correct binary format parsers," in *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. IEEE, 2025, pp. 2535–2547.

[9] M. Zheng, D. Xie, Q. Shi, C. Wang, and X. Zhang, "Validating network protocol parsers with traceable RFC document interpretation," *Proceedings of the ACM on Software Engineering*, vol. 2, no. ISSTA, pp. 1772–1794, 2025.

[10] K. Bhargavan, R. Corin, P.-M. Deniélou, C. Fournet, and J. J. Leifer, "Cryptographic protocol synthesis and verification for multiparty sessions," in *22nd IEEE Computer Security Foundations Symposium (CSF)*, 2009, pp. 124–140.

[11] L. Dong and K. Chen, *Security Analysis Based on Trusted Freshness*. Springer, 2012.

[12] E. Rescorla, "The transport layer security (TLS) protocol version 1.3," Internet Engineering Task Force, RFC 8446, 2018.

[13] J. E. Hopcroft, R. Motwani, and J. D. Ullman, "Introduction to automata theory, languages, and computation," *Acm Sigact News*, vol. 32, no. 1, pp. 60–65, 2001.

[14] D. E. Knuth, "Semantics of context-free languages," *Mathematical systems theory*, vol. 2, no. 2, pp. 127–145, 1968.

[15] D. Harkins and D. Carrel, "RFC2409: The Internet key exchange (IKE)," Internet Engineering Task Force, RFC 2409, 1998.

[16] C. Kaufman, P. Hoffman, Y. Nir, P. Eronen, and T. Kivinen, "Internet key exchange protocol version 2 (IKEv2)," Internet Engineering Task Force, RFC 7296, 2014.

[17] T. Dierks and E. Rescorla, "The transport layer security (TLS) protocol version 1.2," Internet Engineering Task Force, RFC 5246, 2008.

[18] J. Wei, X. Wang, D. Schuurmans, M. Bosma, F. Xia, E. Chi, Q. V. Le, D. Zhou *et al.*, "Chain-of-thought prompting elicits reasoning in large language models," *Advances in neural information processing systems*, vol. 35, pp. 24 824–24 837, 2022.

[19] D. Zhou, N. Schärli, L. Hou, J. Wei, N. Scales, X. Wang, D. Schuurmans, C. Cui, O. Bousquet, Q. Le *et al.*, "Least-to-most prompting enables complex reasoning in large language models," *arXiv preprint arXiv:2205.10625*, 2022.

[20] R. Meng, M. Mirchev, M. Böhme, and A. Roychoudhury, "Large language model guided protocol fuzzing," in *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2024.

[21] Y. Sun, Q. Luo, Y. Wang, Q. Chen, B. Liu, R. Chen, Q. Huang, X. Li, and J. Wang, "SemFuzz: A semantics-aware fuzzing framework for network protocol implementations," in *Proceedings of the ACM Web Conference 2026*, 2026, pp. 3251–3262.

[22] S. Bradner, "Key words for use in RFCs to indicate requirement levels," Internet Engineering Task Force, RFC 2119, 1997.

[23] "GB/T 38636–2020: Information Security Technology — Transport Layer Cryptography Protocol," National Standard of the People's Republic of China, 2020, in Chinese.

[24] E. T. Barr, M. Harman, P. McMinn, M. Shahbaz, and S. Yoo, "The oracle problem in software testing: A survey," *IEEE transactions on software engineering*, vol. 41, no. 5, pp. 507–525, 2014.

[25] V.-T. Pham, M. Böhme, and A. Roychoudhury, "AFLNet: A greybox fuzzer for network protocols," in *Proc. of the 13th IEEE International Conference on Software Testing, Validation and Verification (ICST)*, 2020.

[26] T. Ramananandro, A. Delignat-Lavaud, C. Fournet, N. Swamy, T. Chajed, N. Kobeissi, and J. Protzenko, "EverParse: Verified secure zero-copy parsers for authenticated message formats," in *28th USENIX Security Symposium (USENIX Security 19)*, 2019, pp. 1465–1482.

[27] Kaitai Project, "Kaitai struct: Documentation," https://doc.kaitai.io/, 2026, accessed: 2026-04-28.

[28] P. Biondi, P. Lalet, G. Potter, G. Valadon, and N. Weiss, "Scapy: The Python-based interactive packet manipulation program & library," https://github.com/secdev/scapy, 2024.

[29] J. Somorovsky, "Systematic fuzzing and testing of TLS libraries," in *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*, 2016, pp. 1492–1504.

[30] P. Mandl, "QUIC–UDP-based multiplexed and secure transport," in *TCP, UDP und QUIC Internals: Protokolle und Programmierung*. Springer, 2024, pp. 135–160.

[31] J. A. Donenfeld, "WireGuard: Next generation kernel network tunnel," in *Proceedings of the Network and Distributed System Security Symposium (NDSS)*. The Internet Society, 2017.

[32] C. Aschermann, T. Frassetto, T. Holz, P. Jauernig, A.-R. Sadeghi, and D. Teuchert, "NAUTILUS: Fishing for deep bugs with grammars," in *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2019.

[33] P. Srivastava and M. Payer, "Gramatron: Effective grammar-aware fuzzing," in *Proceedings of the 30th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, 2021, pp. 244–256.

## APPENDIX

*Appendix A*

*Example of Message Structure Grammar:* As shown in Fig. 3, the message structure of TLS 1.3 Finished_Client is illustrated in detail, including field composition and field lengths.

```
{
    "Finished_client": [[{"name": "record_layer"},{"name": "Encrypted_block"}]],
    "Encrypted_block": [[{"name": "handshake_layer"}, {"name": "message_body"}]],
    "message_body": [[{"name": "verify_data"}]],
    "verify_data": [["field_length": "32bits"]]
}
```

Fig. 3: TLS 1.3 Finished_Client message structure fragment.

*Example of Unified Attribute Evaluation System:* As shown in Fig. 4, the attribute annotation results of UAS on the message fields of TLS 1.3 Finished_Client are presented.

```
{
  "Finished_client": [[{"name": "record_layer"},{"name": "Encrypted_block"}]],
  "Encrypted_block": [[{"name": "handshake_layer"},{"name": "message_body"}]],
  "message_body": [[{"name": "verify_data"}]],
  "verify_data": [[{"field_length": "32bits","value": "calculate(truncate(hmac
                  (hkdf_expand_label(client_handshake_traffic_secret, "finished", "", 32),
                  hash(ClientHello_client(handshake_messages)[:] | ...
                  | CertificateVerify_server(handshake_messages)[:]))[0:32])"}]]
}
```

Fig. 4: Attributes of fields in TLS 1.3 Finished_Client.

*Example of Session-Level Cryptographic Computation Grammar:* Fig. 5 and Fig. 6, illustrate the process-oriented and parameter-oriented computation blocks, describing the TLS 1.3 key derivation process and signature algorithm parameters, respectively.

```
KeyDerivation {
    early_secret = calculate(hkdf_extract(repeat(0x00, 32), repeat(0x00, 32)))
    derived_early_secret = calculate(hkdf_expand_label
                            (early_secret, "derived", calculate(hash("")), 32))
    shared_secret = calculate(dh(local(supported_group),
                            ServerHello_server(handshake_messages)[-256:]))
    handshake_secret = calculate(hkdf_extract
                            (derived_early_secret, shared_secret))[0:32]
    hello_transcript_hash = calculate(hash(ClientHello_client(handshake_messages)[:]
                            | ServerHello_server(handshake_messages)[:]))
    client_handshake_traffic_secret = calculate(hkdf_expand_label(handshake_secret,
                             "c hs traffic", hello_transcript_hash, 32))[0:32]
    server_handshake_traffic_secret = calculate(hkdf_expand_label(handshake_secret,
                            "s hs traffic", hello_transcript_hash, 32))[0:32]
    client_handshake_key = calculate(hkdf_expand_label
                            (client_handshake_traffic_secret, "key", "", 16))[0:16]
    client_handshake_iv = calculate(hkdf_expand_label
                            (client_handshake_traffic_secret, "iv", "", 12))[0:12]
    server_handshake_key = calculate(hkdf_expand_labe
                            l(server_handshake_traffic_secret, "key", "", 16))[0:16]
    server_handshake_iv = calculate(hkdf_expand_label
                            (server_handshake_traffic_secret, "iv", "", 12))[0:12]
}
```

Fig. 5: TLS 1.3 Handshake key derivation.

```
Signature Param {
  "padding_type": "PSS","hash_algo": "SHA256","is_prehashed": false,"pss_salt_len": 32
}
```

Fig. 6: TLS 1.3 Signature algorithm block.

*Appendix B*

The prompt templates of the eight steps are presented in detail in Fig. 7 through Fig. 14, following the execution order, where Figs. 7 and 8 correspond to message structure construction, Fig. 9 corresponds to attribute annotation, and Figs. 10 through 14 correspond to computation block definition extraction.

```
@Role: Cryptographic protocol expert. Generate the complete packet composition
for a specified client message based on protocol knowledge and session configuration.
@Instructions:
    @Input variables: ${Session Configuration}, ${Target Message Name}
    @Operation 1: Determine the layered composition, and enumerate all fields and
nested substructures of each layer in the order prescribed by the RFC.
    @Operation 2: Deterministically expand variable-length structures per the session
configuration, annotate bit lengths for terminal fields, split "length prefix + data" into a
length field and a data block, and mark the payloads to be encrypted.
@Output variable: ${Message Layered Structure}
@Format: ..
@Example: ...
```

Fig. 7: Packet composition generation prompt template.

```
@Role: Protocol grammar engineer. Transform the packet composition into grammar
productions, and assign an initial attribute to each terminal field.
@Terminology:
    Value Category: Fixed, Random, Dependent, Length, Local, Calculate
@Instructions:
    @Input variables: ${Message Layered Structure}, ${Session Configuration},
${Reference Example Grammar}
    @Operation 1: Define the start symbol and expand each layer into productions of
subfields, keeping the field order and count fully consistent with the packet composition.
    @Operation 2: Annotate each terminal field with a bit length and a value of one
value category; express variable-length lists as recursive productions.
@Output variable: ${Attribute-Augmented Grammar Structure}
@Format: ...
@Example: ...
```

Fig. 8: Grammar structure generation prompt template.

```
@Role: Grammar validation engineer. Refine the grammar structure so that the value
rule of every field is resolved into references to concrete message fields.
@Terminology:
    Absolute Slice Reference: MessageName_Sender(ProtocolLayer)[start:end]
@Instructions:
    @Input variables: ${Attribute-Augmented Grammar Structure},${Session
Configuration}
    @Operation 1: For each Calculate field, trace all of its inputs through intermediate
variables down to absolute slice references, protocol constants, or concrete
cryptographic primitives.
    @Operation 2: Refine Dependent fields into references to fields of preceding
messages; separate the static and dynamic values of Local fields; merge the plaintext
nodes of encrypted payloads into a unified plaintext block; merge consecutive sub-byte
fields.
    @Operation 3: Preserve the recursive productions of variable-length lists.
@Output variable: ${Refined Grammar Structure}
@Format: ...
@Example: ...
```

Fig. 9: Attribute refinement prompt template.

*Appendix C*

This appendix summarizes the 5 semantic violations detected by SeriCrypt during the experiments. The RFC basis, test construction method, and observation result of each violation are detailed below.

**V-1 No Response to INFORMATIONAL Request**

```
@Role: Cryptographic protocol expert. Derive the base keys and derived keys protecting the handshake into fully expanded formulas.
@Instructions:
    @Input variables: ${Session Configuration}
    @Operation 1: List the keys prescribed by the RFC and write their standard formulas.
    @Operation 2: Recursively expand each formula, substituting operands with fixed values, lengths, random values, previously derived keys, local states, or absolute slice references, and tracing ephemeral keys down to message fields.
    @Operation 3: Apply suite-dependent constraints, e.g., no integrity key is derived for AEAD suites.
@Output variable: ${Key Derivation Formulas}
@Format: ...
@Example: ...
```

Fig. 10: Key derivation formula prompt template.

```
@Role: Protocol cryptography expert. Expand the protocol-specific pseudo-random functions into explicit definitions.
@Instructions:
    @Input variables: ${Key Derivation Formulas}, ${Session Configuration}
    @Operation 1: Identify the protocol-specific PRF constructions invoked by the key derivation formulas, e.g., prf+, P_hash, hkdf_expand_label.
    @Operation 2: Define each construction layer by layer with consistent parameter names, derive the iteration rounds and output lengths from the session configuration, and keep the underlying primitives as generic abstractions.
@Output variable: ${PRF Function Definitions}
@Format: ...
@Example: ...
```

Fig. 11: Encryption flow prompt template.

```
@Role: Decryption flow engineer. Construct the complete decryption chain from the received encrypted payload to the recovered plaintext.
@Instructions:
    @Input variables: ${Key Derivation Formulas}, ${Session Configuration}
    @Operation 1: Specify the parsing of the received payload and the reversal of the encryption chain according to the negotiated algorithm family.
    @Operation 2: Maintain the same session-level state variables as the encryption side, keeping the state machines of both sides consistent.
@Output variable: ${Decryption Formulas}
@Format: ...
@Example: ...
```

Fig. 12: Decryption flow prompt template.

```
@Role: Cryptographic implementation expert. Generate the parameter configuration of the signature algorithm.
@Instructions:
    @Input variables: ${Session Configuration}
    @Operation 1: Determine the padding type, the hash algorithm, whether the signing input is pre-hashed, and the salt length according to the protocol and the session configuration.
@Output variable: ${Signature Parameter Configuration}
@Format: ...
@Example: ...
```

Fig. 13: Signature parameter configuration prompt template.

```
@Role: Protocol cryptography expert. Expand the protocol-specific pseudo-random functions into explicit definitions.
@Instructions:
    @Input variables: ${Key Derivation Formulas}, ${Session Configuration}
    @Operation 1: Identify the protocol-specific PRF constructions invoked by the key derivation formulas, e.g., prf+, P_hash, hkdf_expand_label.
    @Operation 2: Define each construction layer by layer with consistent parameter names, derive the iteration rounds and output lengths from the session configuration, and keep the underlying primitives as generic abstractions.
@Output variable: ${PRF Function Definitions}
@Format: ...
@Example: ...
```

Fig. 14: PRF specialization function prompt template.

RFC Specification: RFC 7296 Section 1: "The types of subsequent exchanges are CREATE_CHILD_SA (which creates a Child SA) and INFORMATIONAL (which deletes an SA, reports error conditions, or does other housekeeping). Every request requires a response."

Test Construction: SeriCrypt first establishes a normal IKEv2 session, then sends a malformed encrypted payload. After strongSwan returns an error and updates its internal state, SeriCrypt continues to send an INFORMATIONAL request.

Observed Result: strongSwan did not return any response to the INFORMATIONAL request. According to RFC 7296, the receiver MUST return a response to every INFORMATIONAL request.

**V-2 Server Selected a Signature Algorithm Not Offered by the Client**

RFC Specification: RFC 8446 Section 4.4.3: "If the CertificateVerify message is sent by a server, the signature algorithm MUST be one offered in the client's 'signature_algorithms' extension unless no valid certificate chain can be produced without unsupported algorithms (see Section 4.2.3)."

Test Construction: SeriCrypt constructs a TLS 1.3 ClientHello message whose signature_algorithms extension does not include the signature algorithm subsequently used by the server.

Observed Result: TLSE still selected a signature algorithm not declared as supported by the client during the handshake and did not abort the handshake.

**V-3 Acceptance of MD5-Signed Certificate**

RFC Specification: RFC 8446 Section 4.4.2.2: "Any endpoint receiving any certificate which it would need to validate using any signature algorithm using an MD5 hash MUST abort the handshake with a 'bad_certificate' alert."

Test Construction: SeriCrypt presents a certificate signed with the MD5 algorithm to TLSE.

Observed Result: TLSE did not send a bad_certificate alert to abort the handshake.

**V-4 Acceptance of ClientHello Missing Required Extension**

RFC Specification: RFC 8446 Section 9.2: "If not containing a 'pre_shared_key' extension, it MUST contain both a 'signature_algorithms' extension and a 'supported_groups' extension. . . . Servers receiving a ClientHello which does not conform to these requirements MUST abort the handshake with a 'missing_extension' alert."

Test Construction: SeriCrypt constructs a non-PSK mode TLS 1.3 ClientHello and removes the signature_algorithms extension, retaining only the supported_groups extension.

Observed Result: TLSE did not send a missing_extension alert, and the handshake continued.

**V-5 Tolerance of Illegal ChangeCipherSpec**

RFC Specification: RFC 8446 Section 5.1: "An implementation which receives any other change_cipher_spec value or which receives a protected change_cipher_spec record MUST abort the handshake with an 'unexpected_message' alert."

TABLE IX: Implementation cost of the nine mutation tasks. Each cell reports LoC / number of functions.

| Task | Target deviation | SeriCrypt | Scapy | TLS-Attacker |
|---|---|---|---|---|
| S1 | client_version = 0x0304 | 1 / 0 | 7 / 0 | 1 / 1 |
| S2 | cipher_suites = [0x00FF] | 1 / 0 | 7 / 0 | 1 / 1 |
| S3 | Finished record length under-counted by 4 | 1 / 0 | 12 / 1 | 5 / 1 |
| D1 | Server Certificate excluded from the transcript hash of verify_data | 1 / 0 | 13 / 1 | 19 / 2 |
| D2 | client_random excluded from the master_secret seed | 1 / 0 | 6 / 1 | 1 / 1 |
| D3 | ClientKeyExchange excluded from the CertificateVerify signature transcript | 1 / 0 | 13 / 1 | 11 / 1 |
| C1 | Finished PRF label swapped | 1 / 0 | 14 / 1 | 1 / 1 |
| C2 | CertificateVerify signature SHA256→SHA1 | 2 / 0 | 8 / 1 | 1 / 1 |
| C3 | GCM tag replaced by the first 16 ciphertext bytes | 1 / 0 | 8 / 1 | 1 / 1 |
| Total | | 10 / 0 | 88 / 7 | 41 / 10 |

Test Construction: SeriCrypt constructs an illegal ChangeCipherSpec message, modifying its payload value from the legal value 0x01 to the illegal value 0x02.

Observed Result: TLSE did not send an unexpected_message alert, but instead ignored the malformed CCS message and continued the handshake.

*Appendix D*

**Experimental Environment.** TLS 1.2, cipher suite TLS_DHE_RSA_WITH_AES_128_GCM_SHA256, mutual authentication, with OpenSSL s_server as the server under test.

**Task Set.** Nine tasks covering three categories—structural fields (S1–S3), cross-message dependencies (D1–D3), and cryptographic computations (C1–C3); the target deviation of each task is identical across the three tools (Table IX).

**Metrics.** The lines of code (LoC) written or modified and the number of functions defined or modified to implement each mutation; the cost of building the baseline client is excluded.

All three tools completed all mutations and generated the corresponding malformed binary messages. The difference lies in the implementation cost: SeriCrypt requires a total of 10 lines and 0 function changes—each task is accomplished by editing 1–2 declaration lines—amounting to 1/4.1 and 1/8.8 of the totals of TLS-Attacker and Scapy, respectively; the difference is largest for dependency-affecting tasks such as D1/D3 (3 lines versus 32 and 31), where imperative tools must rewrite the parsing of the handshake transcript and the hash recomputation logic, whereas SeriCrypt only needs to modify a single concatenation term in the declaration expression.